확장편 · STEP 12~18

# 통화 텍스트로 인입 원인을 보는 관제

## 콜센터 실시간 관제 시스템 확장편 — 음성 인식과 주제·감정 분류

녹취 만들기 · STT · 개인정보 마스킹 · 분류기 평가 · 원인 경보 · AI 공지 초안

기본편(STEP 0~11)의 관제 시스템은 '얼마나 나쁜가'를 보여 준다. 확장편은 통화 내용을 분석해 '무엇 때문에 나쁜가'를 보여 준다. 실습은 Claude 데스크톱 앱의 Code 탭에서 진행한다.

핵심

# 원인을 아는 관제는 인력을 늘리기 전에 인입을 줄인다

기본편 시스템에 통화 텍스트 분석 층을 하나 더 얹는다

1

## 이벤트 데이터는 결과를, 통화 텍스트는 원인을 말한다

서비스레벨이 떨어진 뒤에야 알던 일을 통화 주제의 쏠림으로 먼저 안다.

2

## 받아쓴 텍스트는 오류율을 재고 개인정보를 가린 뒤에 쓴다

STT 오류율(CER)과 마스킹은 분석보다 먼저 확인할 품질 관문이다.

3

## 분류기는 만들 때 보지 않은 데이터로 평가한다

규칙을 만든 대본에서는 100%, 처음 보는 대본에서는 88.9%였다.

**앱 장애 재생에서 원인 경보는 서비스레벨 경보보다 17분 먼저 울렸다**

01

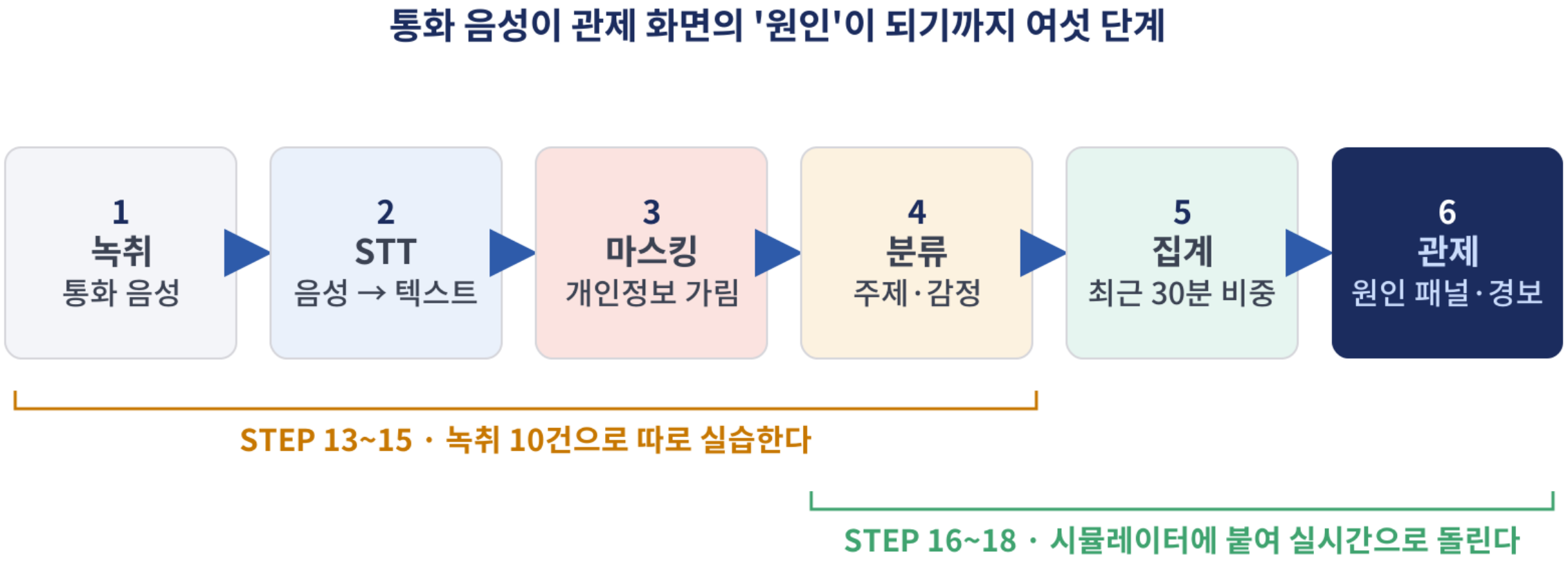
기본 지식

# 통화 음성 분석의 기본 개념

음성이 관제 지표가 되기까지 무엇을 거치고 무엇을 재는지 정리한다

# 통화 음성은 여섯 단계를 거쳐 관제 화면의 원인 정보가 된다

STT(Speech-to-Text) = 음성을 글자로 바꾸는 음성 인식



# 이벤트 데이터와 통화 텍스트는 답하는 질문이 다르다

기본편은 이벤트 데이터만 썼다. 확장편은 통화 텍스트를 더한다

| 구분 | 이벤트 데이터 (기본편) | 통화 텍스트 (확장편) |
|---|---|---|
| 담는 것 | 인입·연결·종료·포기 시각, 상담사 상태 | 고객과 상담사가 한 말 |
| 나오는 곳 | 전화 시스템(CTI·ACD) 로그 | 녹취를 받아쓴 STT 결과 |
| 답하는 질문 | 얼마나 기다렸나, 몇 명이 끊었나 | 왜 전화했나, 얼마나 화가 났나 |
| 관제 지표 | 서비스레벨, 포기율, 점유율 | 주제 비중과 배수, 부정 감정 비율 |
| 주의할 점 | 30분 평균이라 늦게 반응한다 | 인식 오류와 개인정보가 섞인다 |

두 데이터를 함께 보면 '나쁘다'와 '왜 나쁜가'를 한 화면에서 읽는다

# 주제·감정 분류는 규칙에서 시작해 필요할 때 모델로 옮긴다

이 실습은 설명하기 쉬운 규칙 기반 분류기로 만든다

| 방법 | 어떻게 고르나 | 장점 | 한계 |
|---|---|---|---|
| 규칙 기반 | 키워드·표현 사전 점수 | 왜 그렇게 골랐는지 바로 보인다 | 처음 보는 표현에 약하다 |
| 머신러닝 분류 | 라벨 붙은 통화로 학습 | 표현이 달라도 잘 맞힌다 | 라벨 수천 건이 필요하다 |
| LLM 분류 | 주제 정의를 주고 질문 | 라벨 없이도 바로 쓴다 | 통화마다 비용과 지연이 든다 |

## 어느 방법이든 같은 원칙

- 정답 라벨이 있는 데이터를 둘로 나눠, 만들 때 보지 않은 쪽으로 평가한다.

# 통화 텍스트 분석이 관제를 바꾸는 장면은 세 가지다

금융권 AICC에서 STT를 붙이는 이유와 같다

### 원인 모른 채 인력만

**앱·결제 장애**

장애 공지가 늦으면 고객은 계속 전화한다. 상담사를 늘려도 인입이 쌓인다.

*→ 실습: 앱 장애 시나리오*

### 말속의 불만

**부정 감정 쏠림**

같은 대기시간에도 고객의 화는 주제에 따라 다르다. 불만 전담 배정이 필요하다.

*→ 실습: 부정 감정 경보*

### 녹취 속 개인정보

**카드번호·생년월일**

고객은 본인 확인을 위해 번호를 말한다. 화면과 보고서에 그대로 띄우면 안 된다.

*→ 실습: 마스킹*

실습은 세 장면을 각각 시나리오, 경보, 마스킹으로 재현한다

02

확장 설계

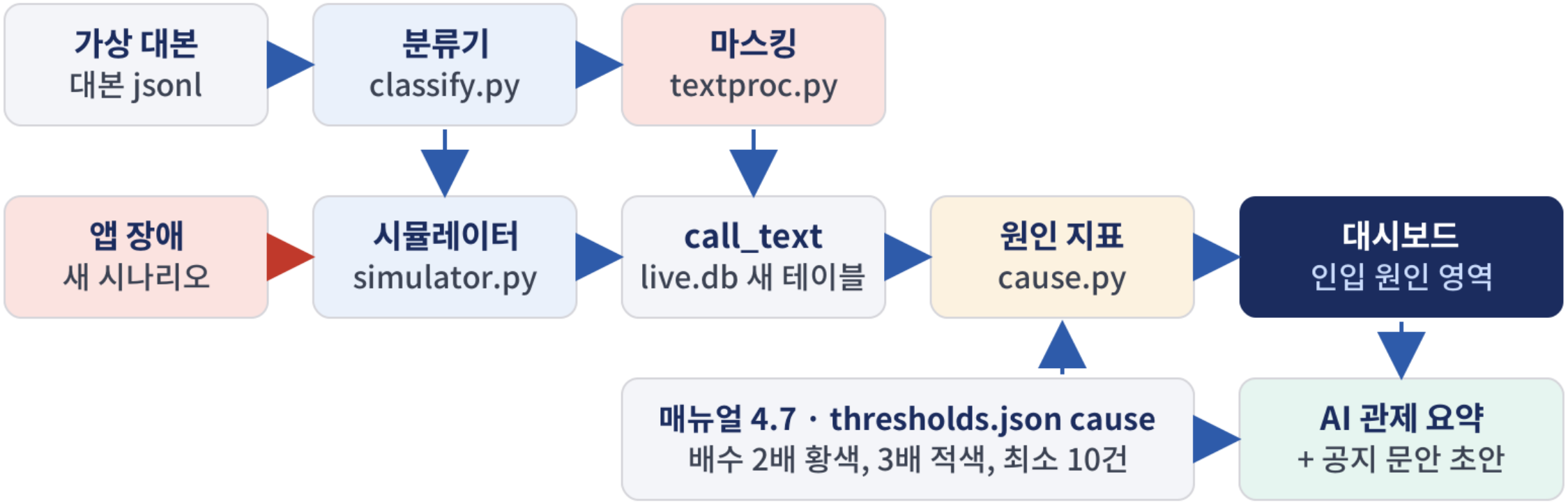
# 확장 구조와 확장 키트

기본편 폴더에 무엇을 더하는지 정한다

# 통화 텍스트는 분류와 마스킹을 거쳐 call_text 테이블에 쌓인다

기본편의 live.db에 테이블 하나, 대시보드에 영역 하나를 더한다

## 확장 구조: 통화 텍스트가 call_text 테이블로 흘러 들어간다



# 확장 키트는 대본 두 묶음과 주제 정의, 새 매뉴얼로 이루어진다

CallCenter_Monitoring_Kit_Extension.zip — 기본편 폴더에 합친다

| 파일 | 내용 | 어디에 쓰나 |
|---|---|---|
| data/call_topics.csv | 주제 8종의 큐, 평소 비중, 대표 키워드 | 분류기, 원인 지표 |
| data/transcripts_seed.jsonl | 가상 대본 120건과 정답 라벨 | 규칙 만들기 |
| data/transcripts_test.jsonl | 처음 보는 표현이 섞인 45건 | 평가 전용 |
| docs/voice_analytics_spec.md | 확장 요구사항 정의서 | 모든 확장 단계 |
| docs/monitoring_manual.md | 4.7 인입 원인 경보를 넣은 새 판 | 경보, AI 요약 |

대본 속 카드번호·전화번호·생년월일은 마스킹 실습용 가짜 번호다

# 주제 8종은 큐마다 평소 비중이 정해져 있다

평소 비중은 큐별 하루 예측 인입과 큐 안 비중으로 계산한다

| 큐 | 주제 | 큐 안 비중 | 평소 비중 | 대표 키워드 |
|---|---|---|---|---|
| 분실·도난 (Q1) | 분실·도난 신고 | 100% | 14.4% | 분실, 도난, 정지 |
| 일반 문의 (Q2) | 결제 승인 오류 | 22% | 14.4% | 승인, 거절, 결제 오류 |
| | 청구·결제일 문의 | 28% | 18.4% | 청구서, 결제일, 연체 |
| | 포인트·혜택 | 20% | 13.1% | 포인트, 적립, 할인 |
| | 해지·재발급 | 15% | 9.8% | 해지, 재발급, 갱신 |
| | 앱 로그인 장애 | 15% | 9.8% | 앱, 로그인, 인증 |
| 대출·한도 (Q3) | 한도 상향 / 카드론·대출 | 50% / 50% | 10.0% / 10.0% | 한도, 상향 / 대출, 금리 |

# 원인 경보는 주제 배수와 부정 감정 비율로 판정한다

매뉴얼 4.7 — 최근 30분 연결 콜 기준, 3분 지속 조건을 건다

$$\text{주제 배수} = \frac{\text{최근 } 30\text{분 주제 비중}}{\text{평소 비중}} \qquad \text{평소 비중}_j = \frac{\text{큐 예측 인입}_{q(j)} \times \text{큐 안 비중}_j}{\sum_k \text{큐 예측 인입}_{q(k)} \times \text{큐 안 비중}_k}$$

| 원인 지표 | 황색 | 적색 | 조건 |
|---|---|---|---|
| 주제 배수 | 평소의 2배 이상 | 평소의 3배 이상 | 그 주제 10건 이상 |
| 부정 감정 비율 | 35% 초과 | 45% 초과 | 전체 10건 이상 |

## 대응 (매뉴얼 4.7)

- 앱·결제 장애 주제가 급증하면 IT 상황실 확인, IVR 장애 멘트, 앱·문자 공지를 요청한다.

시작

# 데스크톱 앱의 Code 탭에서 기본편 폴더를 연다

기본편 STEP 11까지 끝난 callcenter-monitor 폴더에서 이어 간다

### ① 앱 열기

**Claude 데스크톱 앱**

왼쪽 위에서 Code 탭을 고른다.

*구독 계정으로 로그인한 상태*

### ② 폴더 고르기

**callcenter-monitor**

기본편을 만든 폴더를 작업 폴더로 연다.

*확장 키트 zip을 미리 넣어 둔다*

### ③ 붙여 넣기

**STEP 12부터**

앰버 박스의 프롬프트를 입력창에 붙여 넣는다.

*권한 요청은 내용을 읽고 허용*

### ④ 확인하기

**출력과 화면**

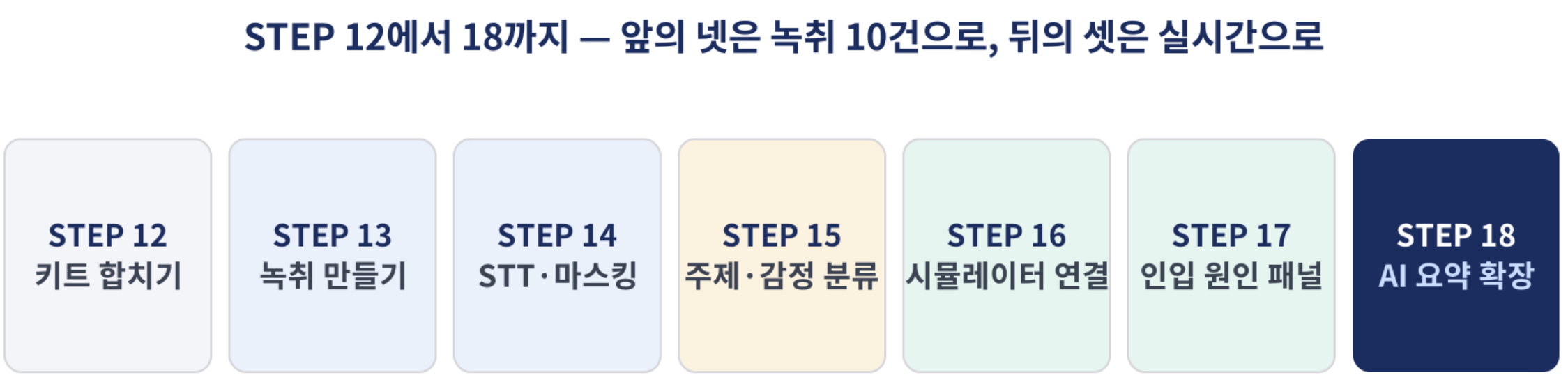
출력 박스와 같은 결과인지 보고, 다르면 되묻는다.

*피드백 칸의 문장을 그대로 써도 된다*

설치·실행·파일 열기·오류 수정은 모두 Code 탭의 Claude에게 시킨다

# 확장편은 일곱 단계로 원인 분석 층을 쌓는다

STEP 13~15는 녹취 10건으로 따로 실습하고, STEP 16~18은 시뮬레이터에 붙여 실시간으로 돌린다




단계마다 실행 결과를 확인하고 git 커밋을 남긴 뒤 넘어간다

03

단계별 구현

# STEP 12~18 프롬프트로 만드는 원인 분석

단계마다 어디 → 입력 → 출력 → 피드백 순서로 진행한다

STEP 12

# 확장 키트를 합치고 대본 데이터부터 들여다본다

대본 120건에는 정답 주제·감정 라벨과 가짜 개인정보가 들어 있다

**① 어디** — Claude 데스크톱 앱 · Code 탭 — callcenter-monitor 폴더를 연 세션

**② [입력] 프롬프트 · 그대로 복사해 붙여 넣기**

```
CallCenter_Monitoring_Kit_Extension.zip을
이 폴더에 풀어 기존 파일과 합쳐 줘.
- docs/monitoring_manual.md는 새 판으로 바꾸고
  이전 판과 달라진 부분을 보여 준다.
- data/transcripts_seed.jsonl을 읽고 주제별·
  감정별 건수와 개인정보가 든 통화 수를 표로
  정리한다.
- CLAUDE.md에 확장 규칙(매뉴얼 4.7, 화면에
  띄우기 전 개인정보 마스킹)을 더하고 커밋한다.
```

**③ [출력] 결과 · 대본 120건 요약**

```
달라진 매뉴얼: 4.7 인입 원인 경보 신설
주제   분실·도난 14   결제 승인 15   청구 18
       포인트 13   해지·재발급 10   앱 로그인 10
       한도 상향 20   카드론 20
감정   중립 59   부정 33   긍정 28
개인정보 포함 통화 35건
  (카드번호·전화번호·생년월일)
[main 7d2c1f0] 확장 키트, CLAUDE.md 규칙 추가
```

**④ 피드백**

- 대본은 상담사와 고객 발화가 번갈아 나오는지, 라벨이 붙어 있는지 확인한다.
- test 파일은 규칙을 만들 때 보지 않는다고 CLAUDE.md에 적었는지 본다.

STEP 13

# 가상 대본을 두 목소리 녹취 파일로 만든다

TTS(Text-to-Speech) — 실제 녹취를 쓸 수 없는 교육 환경에서 녹취를 대신한다

**① 어디** — Claude 데스크톱 앱 · Code 탭 — callcenter-monitor 폴더를 연 세션

**② [입력] 프롬프트 · 그대로 복사해 붙여 넣기**

```
tts_make.py를 만들어 transcripts_test.jsonl
앞 10건을 음성 파일로 만들어 줘.
- edge-tts가 되면 상담사 ko-KR-SunHiNeural,
  고객 ko-KR-InJoonNeural 목소리를 쓴다.
- 안 되면 맥은 say -v Yuna, 윈도는 SAPI
  한국어 음성으로 대신한다.
- 발화 사이에 0.4초 쉼을 넣고 16kHz 모노
  wav로 audio/에 저장한다.
- ffmpeg가 없으면 설치부터 해 줘.
다 되면 첫 파일을 재생해 줘.
```

**③ [출력] 결과 예시 · 실행 로그**

```
ffmpeg 확인 ✓   edge-tts 설치 ✓
저장 audio/W001.wav
저장 audio/W002.wav
...
저장 audio/W010.wav
엔진 edge, 10건 완료

afplay audio/W001.wav 로 재생합니다.
```

**④ 피드백**

- 두 목소리가 번갈아 들리는지, 발화 사이에 쉼이 있는지 귀로 확인한다.
- edge-tts가 막힌 망이면 "맥 say로 다시 만들어 줘"라고 한다.

# 한국어 음성 인식은 단어가 아니라 글자 단위로 오류율을 잰다

WER(단어 오류율)은 조사·띄어쓰기 하나만 틀려도 어절 전체를 오류로 센다

$$\text{문자 오류율}(\text{CER}) = \frac{\underbrace{S}_{\text{바뀐 글자}} + \underbrace{D}_{\text{빠진 글자}} + \underbrace{I}_{\text{더해진 글자}}}{\underbrace{N}_{\text{정답 글자 수}}}$$

정답 “결제일을바꾸고싶어요” → 인식 “결재일을바꾸고시퍼요” $\text{CER} = \frac{3+0+0}{10} = 30\%$

## 읽는 법

- 공백과 문장부호는 빼고 비교한다. '결제'를 '결재'로 받아쓰면 분류 키워드도 함께 놓친다.
- 그래서 분류 규칙에 자주 틀리는 표기(결재, 어플)를 넣거나 STT를 먼저 고친다.

STEP 14

# 받아쓴 텍스트는 오류율을 재고 개인정보를 가린 뒤에 쓴다

STT(Speech-to-Text) = 음성 인식, CER(Character Error Rate) = 문자 오류율

**① 어디** — Claude 데스크톱 앱 · Code 탭 — callcenter-monitor 폴더를 연 세션

**② [입력] 프롬프트 · 그대로 복사해 붙여 넣기**

```
faster-whisper로 stt_eval.py를 만들어 줘.
- audio/의 wav를 small 모델, 한국어로
  받아쓴다.
- 카드번호·전화번호·생년월일을 [카드번호]
  같은 표시로 가린다.
- 대본과 비교해 통화별 CER을 계산한다.
- 결과를 stt_results.csv로 저장하고 평균 CER과
  가장 많이 틀린 단어 5개를 보여 준다.
개인정보가 든 통화로 STT 결과에서도 마스킹이
되는지 따로 확인해 줘.
```

**③ [출력] 결과 형식 · 수치는 환경마다 다르다**

```
call_id  CER    주제(정답/예측)  마스킹
W001    ...%    T3 / T3          -
W004    ...%    T1 / T1          [카드번호] ✓
...
평균 CER ...%   주제 정확도 …%
자주 틀린 단어: 결제→결재 …

stt_results.csv 저장
```

**④ 피드백**

- 숫자가 '구사일공'처럼 한글로 나오면 마스킹이 빠진다. "한글 숫자도 가려 줘"
- STT 텍스트에는 화자 구분이 없다. 상담사 말이 감정 판정에 섞이는지 본다.

STEP 15

# 분류기는 만들 때 보지 않은 테스트 데이터로 평가한다

seed로 규칙을 만들고, 처음 보는 표현이 섞인 test로 실력을 잰다

**① 어디** — Claude 데스크톱 앱 · Code 탭 — callcenter-monitor 폴더를 연 세션

**② [입력] 프롬프트 · 그대로 복사해 붙여 넣기**

```
classify.py를 만들어 줘. 고객 발화만 쓴다.
- 주제: call_topics.csv 키워드 점수가
  가장 높은 주제를 고른다.
- 감정: 부정·긍정 표현 사전으로 고른다.
먼저 seed로 정확도와 오분류 표를 보여 줘.
그다음 오분류 문장을 보고 추가할 표현을
제안해 줘. 내가 승인하면 반영하고,
반영 전후 정확도를 seed와 test에서 비교해 줘.
표현을 고를 때 test 파일은 보지 마.
```

**③ [출력] 결과 · python classify.py**

```
                주제 seed   주제 test
v1 키워드만      76.7%       73.3%
v2 표현 추가    100.0%       88.9%
감정 정확도     100.0%      100.0%

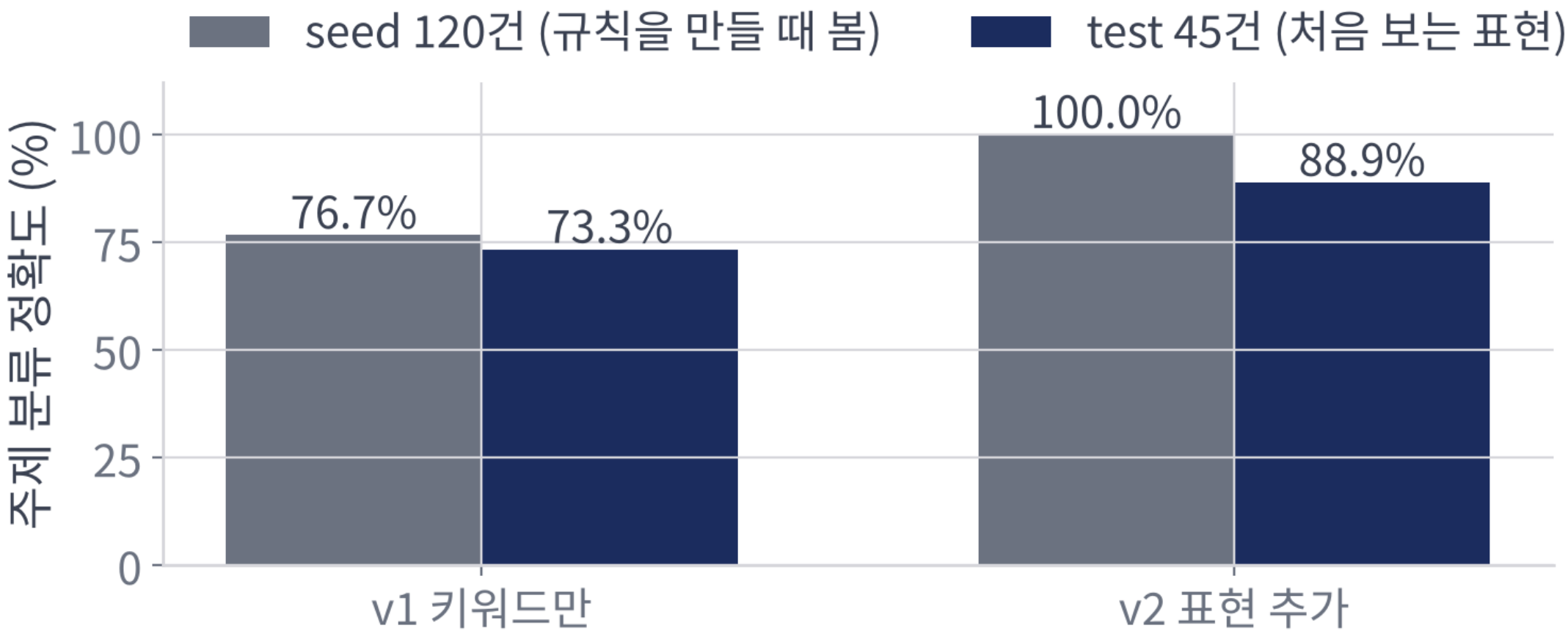
추가 표현 예: 어플, 튕겨, 결재,
빠져나가, 빌린, 늘려, 망가져
```

**④ 피드백**

- seed 100%는 외운 결과다. 보고할 실력은 test 88.9%다.
- "test에서 틀린 문장을 보여 주되 규칙에는 넣지 마"로 한계를 확인한다.

# 분류기 정확도는 처음 보는 대본에서 88.9%로 내려갔다

규칙을 만든 seed에서는 100% — 감정은 두 데이터 모두 100%(가상 대본이 단순해서 쉽다)



STEP 16

# 콜마다 통화 텍스트를 붙여 실시간 원인 데이터를 만든다

노트북에서 통화마다 STT를 돌리지 않고, 대본 텍스트를 STT 결과로 간주한다

**① 어디** — Claude 데스크톱 앱 · Code 탭 — callcenter-monitor 폴더를 연 세션

**② [입력] 프롬프트 · 그대로 복사해 붙여 넣기**

```
simulator.py를 확장해 줘.
- 콜이 들어오면 call_topics.csv 비중으로
  주제를 정한다.
- 연결되면 그 주제 대본 하나를 classify로
  분류해 call_text 테이블에 쓴다(시각, 큐,
  정답·예측 주제, 감정, 마스킹한 앞 40자).
- 새 시나리오 '앱 장애': 일반 문의 큐에 앱
  로그인 장애 콜이 평소의 60%만큼 더 온다.
--speed 200으로 14:00 앱 장애, 15:00 정상을
넣고 13:50·14:10·14:30·15:00 상황을 표로 줘.
```

**③ [출력] 결과 · 앱 장애 하루 재생**

| 시각 | 1위 주제(비중, 배수) | 부정 | SL |
|---|---|---|---|
| 13:50 | 해지·재발급 11% 1.2배 | 27% | 68% |
| 14:10 | 앱 로그인 21% 2.2배 | 31% | 98% |
| 14:30 | 앱 로그인 40% 4.0배 | 46% | 77% |
| 15:00 | 앱 로그인 44% 4.5배 | 43% | 83% |

※ 시뮬레이션 결과, 실행마다 조금 다르다

**④ 피드백**

- 14:10에 SL은 98%인데 주제 비중은 이미 2배다. 이 차이가 확장편의 핵심이다.
- 포기 콜은 통화 텍스트가 없다. 원인 지표는 연결 콜만 센다는 점을 확인한다.

STEP 17

# 원인 경보는 서비스레벨 경보보다 17분 먼저 울렸다

주제 배수 = 최근 30분 비중 ÷ 평소 비중, 기준은 매뉴얼 4.7과 thresholds.json

**① 어디** — Claude 데스크톱 앱 · Code 탭 — callcenter-monitor 폴더를 연 세션

**② [입력] 프롬프트 · 그대로 복사해 붙여 넣기**

```
cause.py와 대시보드 '인입 원인' 영역을 만들어 줘.
- 최근 30분 주제 비중을 평소 비중과 나란히
  막대로 보인다.
- 부정 감정 비율, 가장 늘어난 주제와 배수
- 매뉴얼 4.7 원인 경보에 3분 지속 조건을
  건다. 기준값은 thresholds.json의 cause에 둔다.
- 최근 통화 4건을 마스킹된 문장으로 보인다.
사이드바에 '앱 장애' 버튼을 더하고, 하루 재생에서
원인 경보와 SL 경보가 처음 울린 시각을 비교해 줘.
```

**③ [출력] 결과 · 첫 경보 시각 (14:00 앱 장애)**

```
14:12  앱 로그인 장애 급증  황색 (2.2배)
14:15  부정 감정            황색
14:21  앱 로그인 장애 급증  적색
14:29  서비스레벨           황색

원인 경보가 SL 경보보다 17분 빠름
09:00~14:00 정상 구간 원인 경보 0건
```

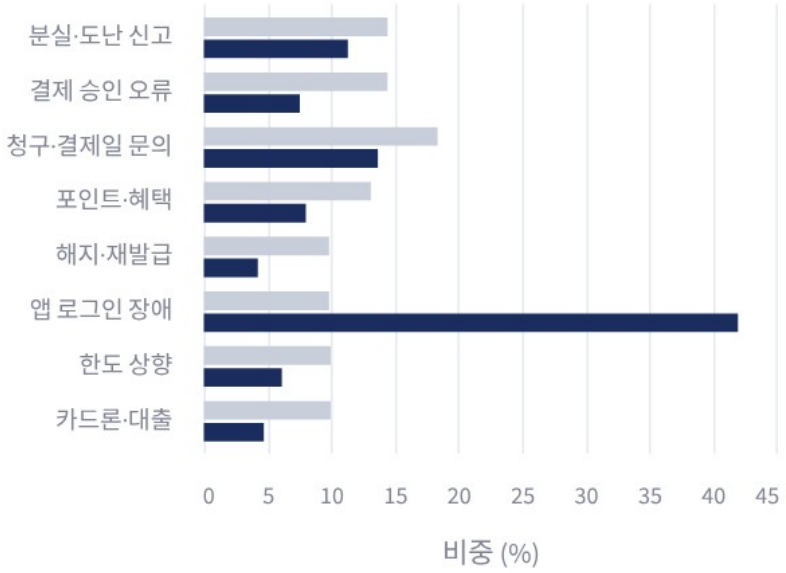
**④ 피드백**

- 정상 구간에 원인 경보가 울리면 오경보다. "오경보 시각과 그때 비중을 보여 줘"
- 건수가 적은 주제는 배수가 크게 튄다. 최소 10건 조건이 있는지 확인한다.

# 인입 원인 영역은 앱 로그인 장애가 평소의 네 배라고 알려 준다

앱 장애 주입 후 가상 시각 14:41 — 막대: 최근 30분(네이비)과 평소(회색)

**인입 원인 — 통화 텍스트 분석** (최근 30분 연결 212건)




구분
- 평소
- 최근 30분

부정 감정 비율

45%

가장 늘어난 주제

앱 로그인 장애

↑ 평소의 4.3배

**원인 경보** (3분 이상 유지)

- **RED** 앱 로그인 장애 급증 평소의 4.3배 (89건)
- **YELLOW** 부정 감정 45%

**최근 통화 (개인정보 마스킹)**

[앱 로그인 장애 · 부정] 앱 로그인이 안 돼요. 너무 불편해서 화가 나요. 제 번호는 [전화번호]

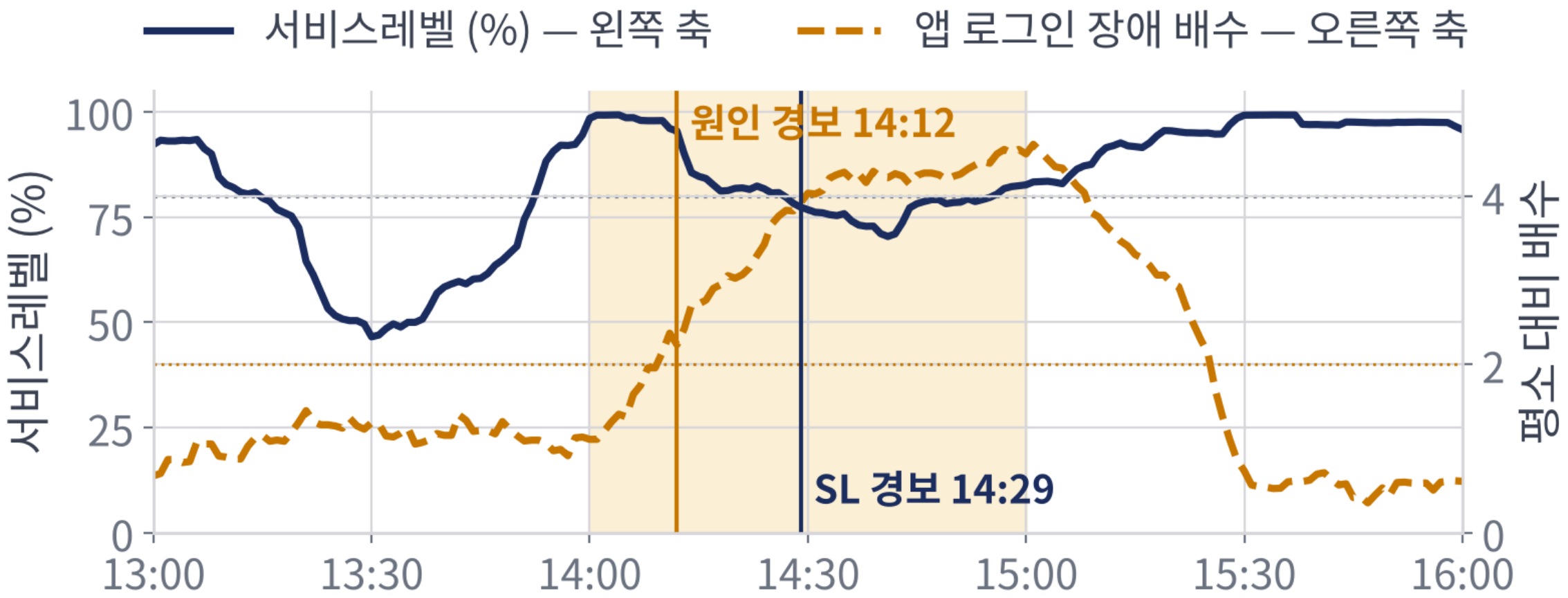
[청구·결제일 문의 · 중립] 이번 달 결재 금액이 맞는지 봐 주세요. 지난달보다 두 배 가까이 나왔어

[앱 로그인 장애 · 부정] 앱 로그인이 안 돼요. 너무 불편해서 화가 나요. 제 번호는 [전화번호]

[앱 로그인 장애 · 중립] 어플 업데이트하고 나서 화면이 하얘요. 제 번호는 [전화번호]이에요. 아

# 주제 배수는 서비스레벨보다 먼저 움직인다

14:00 앱 장애 주입 — 원인 경보 14:12, 서비스레벨 경보 14:29 (시뮬레이션)



# AI 요약이 원인과 공지 문안 초안까지 낸다

원인 경보가 있으면 매뉴얼 4.7-3에 따라 고객 공지 문안 초안 2문장을 받는다

**① 어디** — Claude 데스크톱 앱 · Code 탭 — callcenter-monitor 폴더를 연 세션

**② [입력] 프롬프트 · 그대로 복사해 붙여 넣기**

```
ai_summary.py가 인입 원인(주제 비중과 배수,
부정 감정 비율, 원인 경보)도 받게 고쳐 줘.
- 원인 경보가 있으면 매뉴얼 4.7-3에 따라
  고객 공지 문안 초안 2문장을 덧붙인다.
- 조치마다 절차 번호를 단다.
14:30 앱 장애 상황으로 실행해 결과를 보여 줘.
시나리오 이름은 넘기지 마. AI가 원인을
데이터로 찾는지 보려는 것이다.
```

**③ [출력] 결과 · claude -p 실제 응답을 줄인 것**

```
1) 인입의 40%가 앱 로그인 장애, 평소 4.0배
   (4.7-1). 부정 감정 46%로 45% 초과
2) ① IT 상황실에 장애 확인 (4.7-2, 4.7-5)
   ② 불만 전담 배정, 콜백 안내 (4.7-4, 4.2)
   ③ IVR 장애 멘트, 앱·문자 공지 (4.7-3)
4) 공지 초안: 현재 일부 고객님께서 앱 로그인이
   원활하지 않아 확인하고 있습니다. …
```

**④ 피드백**

- 시나리오 이름 없이도 원인을 앱 로그인 장애로 짚었는지 본다.
- 공지 문안에 확인되지 않은 원인(해킹 등)을 쓰면 "확인된 사실만 써 줘"라고 한다.

# 관제의 질문이 '얼마나'에서 '왜'로 넓어졌다

확장편에서 직접 확인한 것

**1**

## 원인 경보가 서비스레벨 경보보다 17분 빨랐다

인력을 늘리기 전에 IT 확인과 공지로 인입 자체를 줄일 시간을 번다.

**2**

## 분류기 실력은 test 정확도로 말한다

seed 100%는 외운 결과이고, 처음 보는 표현에서는 88.9%였다.

**3**

## 텍스트는 마스킹한 뒤에만 화면에 띄운다

카드번호·전화번호·생년월일을 가리고, 한글로 읽힌 숫자까지 챙긴다.

**다음으로 해 볼 것 — 실제 녹취로 STT 정확도 측정, 화자 분리, LLM 분류와의 비교**